A Versioned Experimental Platform for Large-Scale Database Benchmarking

Anonymous Authors

Beihang University, Beijing, China

Abstract. Database experimentation often repeats the same expensive lifecycle: importing a large dataset, deploying a clean instance, running a workload, and recording which code revision or configuration produced the result. Existing tools cover only slices of this process. BenchBase, OLTPBench, and HammerDB are strong workload runners; DVC and lakeFS snapshot and roll back data; Oracle Real Application Testing and SQL Server Distributed Replay preserve real traces. What is still missing is a versioned experimental platform that treats import, deployment, testing, and comparison as one reusable workflow. We present a lightweight database testing platform that packages imported datasets into reusable snapshots, exposes five user actions (import, archive, deploy, run, compare), and records every run with snapshot, workload, configuration, and Git commit tags. Read-only experiments can reuse an existing deployment, while update-heavy experiments can be restored to a clean snapshot immediately after execution. The demo walks through snapshot creation, deployment reuse, rerun recovery, and history-based comparison.

Keywords: database testing · version control · benchmark · large-scale data

1 Introduction and Motivation

Benchmarking has become a mature practice in database systems, yet the surrounding experiment workflow is still surprisingly manual. A typical study begins with a large data import, then proceeds to a workload run, and finally ends with ad hoc notes about which Git commit, schema version, or configuration was used. This is acceptable for small datasets, but it becomes expensive and error prone once the data reaches hundreds of gigabytes and the system is revised frequently. For a demo audience, the most important question is not only whether a benchmark can run, but whether the same imported data can be reused, redeployed, and compared without repeating the most expensive step every time.

Existing tools cover only part of the lifecycle. Benchmark frameworks such as BenchBase and OLTPBench focus on running workloads against relational DBMSs. HammerDB adds industrial-strength support for schema building and timed workloads. Versioning systems such as DVC and lakeFS are excellent at snapshotting large datasets, but they do not execute database workloads. Replay

systems such as Oracle Real Application Testing and SQL Server Distributed Replay preserve real traces, but they are tied to specific database ecosystems and are not designed as a general purpose platform for repeated, versioned benchmark experiments. Our goal is to connect these pieces into a single, reusable platform for large-scale database testing.

Contributions. The demo makes three practical contributions:

- A large-data import pipeline that turns a one-time ingest into a reusable snapshot artifact.
- A decoupled workflow that separates import, deployment, and testing so that read-only runs can reuse an existing deployment, while update-heavy runs can be restored quickly.
- A history-based experiment record that links results to snapshot, workload, configuration, and Git commits, making regression inspection and comparison much easier.

2 Platform Design and Prototype Status

Figure 1 summarizes the platform architecture. The platform is organized around four durable objects. The *snapshot archive* stores the imported dataset once and makes it reusable. The *deployment record* points to the database instance restored from a snapshot. The *workload bundle* captures the benchmark or test script selected for a run. The *run log* stores the outcome together with the snapshot, configuration, and Git commit identifiers. In other words, every experiment is treated as a record that can be replayed or compared later.

The prototype exposes five user-facing actions: *import*, *archive*, *deploy*, *run*, and *compare*. Import loads the raw dataset into the target DBMS. Archive freezes the loaded state into a reusable snapshot. Deploy restores that snapshot into a clean environment when needed. Run executes the selected workload and stores the measured outcome. Compare opens the run history and contrasts two records by snapshot, workload, configuration, or Git commit. This action set is intentionally small so the demo can be explained and remembered quickly.

For each run, the platform records the snapshot identifier, workload identifier, DBMS configuration fingerprint, Git commit, start and end time, and a compact result summary. This is sufficient to answer the two questions that typically matter during database experimentation: what changed, and where did the change come from?

This design is deliberately simple. The goal is not to build a new DBMS, but to reduce the overhead of repeated experiments. If a dataset is imported once and materialized as a stable snapshot, later benchmark runs can start from that snapshot instead of replaying the ingest step every time. This matters most when several workload variants must run over the same base data.

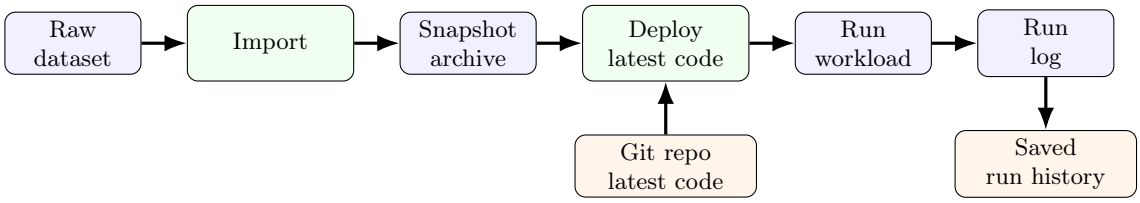
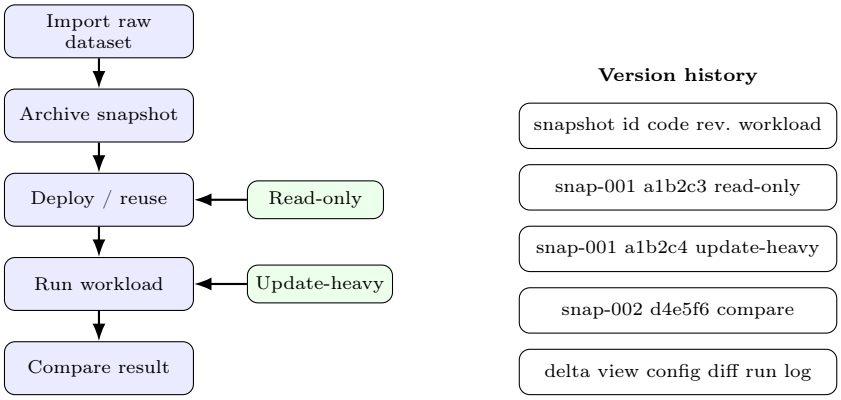


Fig. 1. Platform architecture. The final version will replace this box with the actual system diagram for import, deployment, Git-backed code refresh, and run history.



(a) Import once, deploy many times (b) History-based run records

Fig. 2. Demo walkthrough placeholder. Two panels should eventually show the main user journey and the version-history view.

3 Demo Walkthrough

Figure 2 shows the main user journey. The demo is structured around three scenarios, and each one is designed to answer a practical question that a database engineer would ask when trying to reproduce or compare experiments.

3.1 Scenario 1: Import once, test many times

The first scenario shows how a large dataset is ingested only once. After the ingest finishes, the platform packages the resulting state into a snapshot artifact and registers it in the archive. Multiple benchmark runs can then reuse this snapshot without re-running the full import. This is the core benefit for large datasets: the expensive step is paid only once, while later experiments remain reproducible and directly comparable.

3.2 Scenario 2: Read-only and update-heavy workloads

The second scenario compares two workload types. Read-only workloads can be executed directly on a deployed snapshot with no additional recovery step.

Update-heavy workloads, on the other hand, are isolated so that the platform can restore the clean snapshot after the run. This makes it possible to test both query-only and modifying workloads without manually rebuilding the database after every experiment. In the demo, this is the part where the audience can see the difference between a reusable deployment and a disposable execution environment.

3.3 Scenario 3: History-based comparison

The third scenario highlights the run history. Two runs can be compared side by side using the tags attached to their data, Git commit, and configuration. The demo will show how a user can inspect whether a change comes from the workload, the dataset, the DBMS configuration, or the platform code. This is the main reason we frame the system as a *history-aware* testing platform rather than just a benchmark launcher. The comparison view is also where regression inspection becomes practical, because the user no longer has to guess which component changed between two measurements.

4 Positioning and Related Work

Table 1 summarizes the gap we are targeting. The important point is not that the existing systems are weak; rather, each of them is optimized for one slice of the problem. Benchmark runners are good at executing workloads. Versioning tools are good at snapshotting data. Replay tools are good at preserving traces. Our platform links these slices into one lifecycle so that large-data database experiments can be repeated, compared, and rolled back with much less overhead.

For the demo presentation, we will emphasize two concrete views: the platform dashboard that lists snapshots, deployments, and experiment runs, and the version history view that makes it easy to inspect performance changes. This is the part most useful to users who currently spend too much time on data preparation rather than on the experiment itself. The related-work section can stay short, because the demo value comes mainly from the workflow and the prototype.

5 Conclusion

We proposed a lightweight versioned experimental platform for large-scale database benchmarking. The platform is intentionally focused on the experiment lifecycle: import once, package as a snapshot, deploy on demand, run workloads, and compare by history. The demo shows why such a platform is useful even in a mature relational database ecosystem: mature DBMSs still leave users with repeated imports, ad hoc deployment, and version ambiguity. In the final version, we will replace the placeholder figures with the actual architecture diagram, workflow screenshot, and history view. The key point is simple: the platform makes repeat work visible, reusable, and easier to compare after one walkthrough.

Table 1. Positioning against existing tools.

Tool family	Main gap relative to our platform
BenchBase / OLTPBench / HammerDB	Strong at workload execution, but they do not package imported data into reusable snapshots or make deployment a reusable experiment artifact.
DVC / lakeFS	Strong at data versioning and rollback, but they do not execute database workloads or compare benchmark runs.
Oracle / SQL Server replay	Strong at capture-replay, but they are vendor-centric and not designed as a general benchmark platform.
Our platform	Unifies large-data import, snapshot reuse, deployment/test decoupling, and history-based comparison.

References

1. M. L. Kersten, P. Koutsourakis, S. Manegold, and Y. Zhang: SQALPEL: A database performance platform. CIDR demo paper (2019).
2. BenchBase project homepage, `https://github.com/cmu-db/benchbase`, last accessed 2026/04/19.
3. D. E. Difallah, A. Pavlo, C. Curino, and P. Cudré-Mauroux: OLTP-Bench: An extensible testbed for benchmarking relational databases. PVLDB **7**(4), 277–288 (2013).
4. HammerDB documentation, `https://www.hammerdb.com/docs4.11/index.html`, last accessed 2026/04/19.
5. DVC documentation: Versioning data and models, `https://doc.dvc.org/example-scenarios/versioning-data-and-models`, last accessed 2026/04/19.
6. lakeFS homepage, `https://lakefs.io/`, last accessed 2026/04/19.
7. Oracle Real Application Testing documentation, `https://docs.oracle.com/en-us/iaas/autonomous-database-serverless/doc/autonomous-real-application-testing.html`, last accessed 2026/04/19.
8. Microsoft SQL Server Distributed Replay overview, `https://learn.microsoft.com/en-us/sql/tools/distributed-replay/sql-server-distributed-replay?view=sql-server-ver17`, last accessed 2026/04/19.